R1.01 - Initiation au Développement : Variables, Types de Données, Allocation Mémoire et Structures Conditionnelles

Thème principal : Programmation impérative, modèle mémoire RAM et structures de contrôle

Synthèse exécutive : Ce cours pose les fondations fondamentales du développement en BUT 1 Informatique. L'objectif est de comprendre comment un ordinateur manipule la mémoire vive (RAM) à travers le concept de variable, pourquoi le typage rigoureux est indispensable pour éviter les dépassements de capacité (overflow), comment la norme IEEE 754 impacte les calculs sur les flottants, et comment piloter l'exécution séquentielle via des structures conditionnelles optimisées par l'évaluation paresseuse (short-circuit).

Le concept de Variable et le Modèle Mémoire (RAM)

Pour qu'un programme puisse traiter des informations, il doit les conserver temporairement en mémoire vive (RAM). La variable est l'abstraction fondamentale permettant au développeur de manipuler cette mémoire sans gérer des adresses hexadécimales. Elle réunit quatre propriétés : un nom symbolique, une adresse mémoire physique, un type (qui définit la taille en octets) et une valeur binaire actuelle. Lors d'une affectation, la valeur précédente est irrémédiablement écrasée.

- Variable : Abstraction matérielle représentant un emplacement réservé en RAM.
- Affectation : Stockage d'une valeur binaire écrasant l'ancienne.
- Adresse Mémoire : Identifiant physique unique (ex: 0x7FFF5FBFF05C).
- Typage : Spécification du nombre d'octets et mode d'interprétation des bits.

Cas concret enseignant : *Analogie du casier dans le vestiaire de l'IUT : Chaque casier a un numéro fixe (l'adresse mémoire). Vous collez une étiquette 'age' (le nom). La taille du casier détermine ce qu'on peut y mettre (le type). Mettre un sac rouge (valeur 20) puis un sac bleu (valeur 21) fait disparaître le sac rouge. Mettre un objet trop grand provoque un débordement (overflow).*

Les Types Primitifs, Représentation Binaire et Gestion du Dépassement (Overflow)

La capacité d'un type est limitée par son nombre de bits. Pour les entiers, un dépassement de la borne supérieure fait basculer la valeur vers la limite négative (overflow). Pour les flottants, la norme IEEE 754 génère des approximations (ex: 0.1 + 0.2 donne 0.30000000000000004), ce qui impose d'utiliser un seuil epsilon pour les comparaisons.

- Types Entiers : Nombres sans virgule (ex: int 32 bits de -2.1B à +2.1B).
- Overflow : Dépassement de capacité entraînant un bouclage vers les négatifs.
- Types Flottants : Approximation selon la norme IEEE 754.
- Interdiction d'égalité stricte : Ne jamais utiliser '==' entre deux floats.

Cas concret enseignant : *L'anecdote de la fusée Ariane 5 (1996) : Un overflow lors de la conversion d'un flottant 64 bits en un entier 16 bits a provoqué le crash de la fusée. Règle d'or : ne jamais utiliser 'float' pour l'argent, mais des entiers en centimes.*

Structures Conditionnelles et Évaluation Paresseuse (Short-Circuit)

Les structures conditionnelles permettent de prendre des décisions via des expressions booléennes. L'évaluation paresseuse (short-circuit) optimise l'exécution : dans '(A && B)', si A est faux, B n'est pas évalué. Cela évite aussi des erreurs fatales comme la lecture de pointeurs nuls.

- Exécution Séquentielle : Déroulement ligne par ligne par défaut.
- Bloc Conditionnel : SI / SINON SI / SINON pour dévier le flux.

• Évaluation Paresseuse : Arrêt dès que le résultat logique est garanti.

Cas concret enseignant : *Contrôle d'accès : 'SI (utilisateur_existe ET mot_de_passe_correct)'. Si l'utilisateur n'existe pas (faux), le système ne cherche pas le mot de passe, ce qui évite un plantage de type NullPointer.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi 0.1 + 0.2 n'est pas exactement égal à 0.3 quand on l'affiche en programmation ?
Réponse du professeur : Parce que les ordinateurs stockent les nombres flottants en binaire (norme IEEE 754). Tout comme 1/3 ne s'écrit pas avec un nombre fini de chiffres en base 10, certains nombres décimaux ont une écriture infinie en base 2, ce qui crée de légères erreurs d'arrondi.

Question étudiant : Est-ce qu'une variable déclarée à l'intérieur d'un bloc SI existe toujours après la fin du bloc SINON ?
Réponse du professeur : Cela dépend de la portée (scope) des variables dans le langage utilisé (comme le C ou Java), mais par bonne pratique d'architecture, il est toujours préférable de déclarer ses variables en amont du bloc d'instructions pour éviter qu'elles ne soient détruites à la sortie de l'accolade.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme d'un programme qui demande l'âge d'un utilisateur et son statut d'étudiant (Booléen), puis calcule le prix d'un billet de cinéma sachant que : Tarif plein = 10.00 €, Mineurs (< 18 ans) = 6.00 €, Étudiants majeurs (>= 18 ans et étudiant) = 7.50 €, Seniors (>= 65 ans) = 5.00 €.

Résolution pas à pas :
Étape 1 : Déclarer age (Entier), estEtudiant (Booléen), prix (Flottant). Étape 2 : Saisir age et estEtudiant. Étape 3 : Évaluer les conditions du plus restrictif au plus général : - SI (age < 18) ALORS prix = 6.00 - SINON SI (age >= 65) ALORS prix = 5.00 - SINON SI (estEtudiant == VRAI) ALORS prix = 7.50 - SINON prix = 10.00 Étape 4 : Afficher le prix.

Résultat : Pour un étudiant de 20 ans, 'age < 18' est faux, 'age >= 65' est faux, 'estEtudiant == VRAI' est vrai -> Le prix est 7.50 €. L'ordre garantit qu'un mineur étudiant paiera bien 6.00 €.

■ Auto-Évaluation Flash d'Examen

Q : Que se produit-il en cas d'overflow sur un entier signé ?

R : Le bit de poids fort bascule et la valeur rebondit brusquement vers la limite minimale négative.

Q : Pourquoi l'évaluation paresseuse (short-circuit) est-elle utile dans 'SI (A && B)' si A est faux ?

R : Elle stoppe l'évaluation immédiatement sans tester B, ce qui optimise le temps de calcul et évite des erreurs d'exécution (ex: accès à un objet nul).

Q : Quelle est la règle absolue pour comparer deux flottants ?

R : Ne jamais utiliser l'opérateur '==' en raison des erreurs d'arrondi IEEE 754, mais vérifier si la valeur absolue de leur différence est inférieure à un epsilon.